

LỄ BẢO VỆ ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ THÔNG TIN - 7480201

TÊN ĐỀ TÀI:

XÂY DỰNG HỆ THỐNG DỊCH GIỌNG NÓI ĐA NGÔN NGỮ GẦN THỜI GIAN THỰC
SỬ DỤNG MÔ HÌNH AI CHẠY CỤC BỘ

A Near Real-Time Speech Translation System Using Locally Hosted AI Models

GVHD:

ThS. Nguyễn Thành Luân

NGƯỜI THỰC HIỆN:

24410300 - Ngô Mạnh Hùng

Thành phố Hồ Chí Minh, ngày 10 tháng 10 năm 2026

Nội dung

I. Tổng quan đề tài

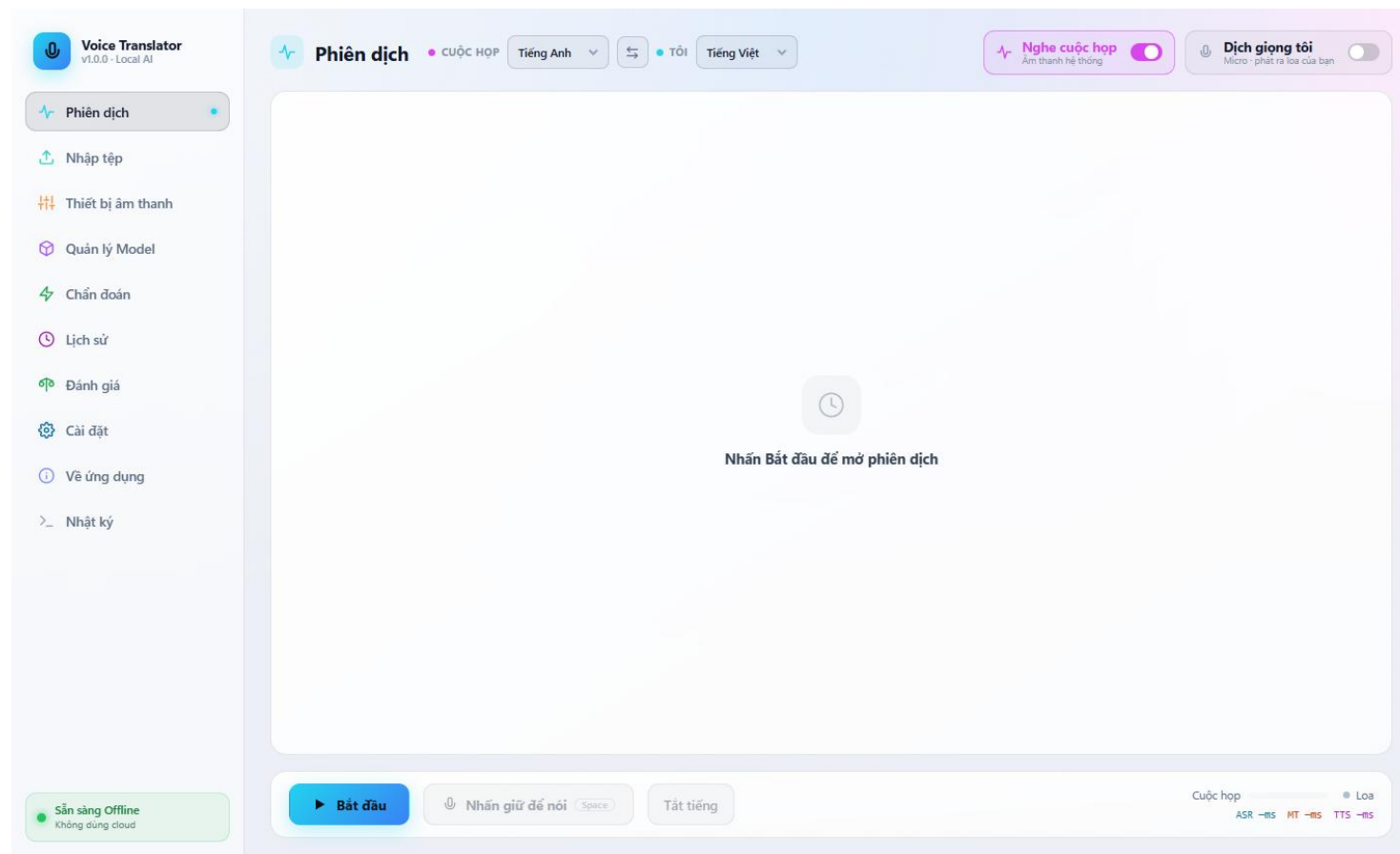
1. Bài toán và các giải pháp liên quan
2. Mục tiêu và phạm vi

II. Phương pháp thực hiện

1. Chuỗi xử lý và lựa chọn mô hình
2. Kiến trúc hệ thống
3. Các kỹ thuật xử lý chính
4. Ứng dụng, đóng gói và kiểm thử

III. Kết quả thực nghiệm

IV. Kết luận



Màn hình Phiên dịch của ứng dụng

I. Tổng quan đề tài

1. Bài toán

Họp và học trực tuyến với người nói ngôn ngữ khác ngày càng phổ biến (Google Meet, Teams, Zoom). Các công cụ dịch giọng nói hiện có gần như đều chạy trên đám mây, kéo theo ba vấn đề:



Phụ thuộc đường truyền

Mạng chậm chờn thì bản dịch trễ hoặc mất giữa cuộc họp.



Tổn phí theo lượt dùng

Tính theo thời lượng hoặc gói thuê bao; nhiều tính năng chỉ có ở gói trả phí.



Nội dung rời khỏi máy

Toàn bộ âm thanh cuộc họp phải gửi tới máy chủ của bên thứ ba.

Trong khi đó, mô hình mã nguồn mở cho từng khâu đã đủ tốt để chạy trên máy cá nhân: **Whisper** nhận dạng ~100 ngôn ngữ [19], **NLLB-200** dịch trực tiếp 200 ngôn ngữ [4], **sherpa-onnx** tổng hợp giọng nói offline [12].

Câu hỏi của đề tài: một laptop cá nhân có chạy được trọn chuỗi dịch giọng nói, đủ nhanh để theo kịp người nói, mà không cần đám mây?

I. Tổng quan đề tài

1. Các giải pháp liên quan

Tiêu chí	Google Meet [7]	SeamlessM4T [23]	TranscriptionSuite [10]	LocalVocal [22]	Đề tài
Chạy cục bộ, không cần mạng	Không	Có	Có	Có	Có
Có tiếng Việt	Không	Có	Có	Có	Có
Dịch	Có	Có	Không	Có	Có
Tổng hợp giọng nói	Có	Có	Không	Không	Có
Thu âm thanh cuộc gọi	Có	Không	Không	Không	Có
Ứng dụng desktop độc lập	Không	Không	Có	Không	Có
Đo được lỗi từng khâu	Không	Không	Không áp dụng	Không	Có

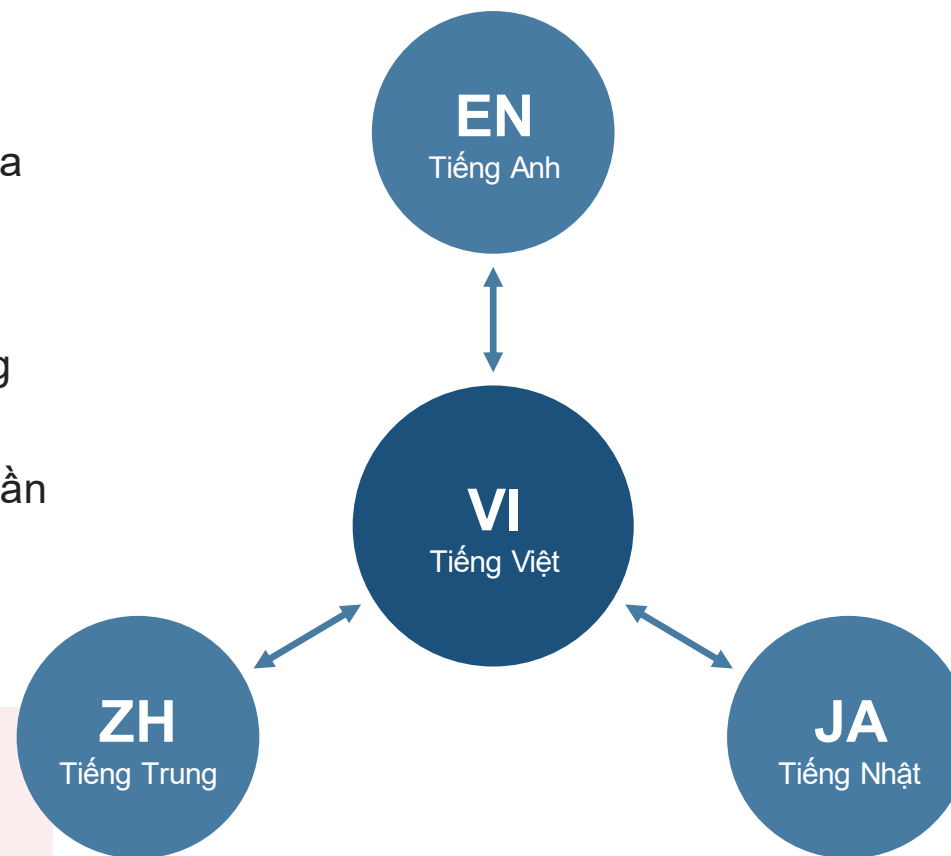
Chưa có giải pháp nào vừa chạy hoàn toàn cục bộ, vừa có tiếng Việt, vừa đủ bốn khâu thu, nghe, dịch và đọc cho cả hai phía của một cuộc gọi. Mô hình của các dịch vụ đám mây lớn hơn nhiều bậc, nên mục tiêu của đề tài là đủ dùng trên phần cứng phổ thông, có số đo cho từng khâu.

I. Tổng quan đề tài

2. Mục tiêu và phạm vi

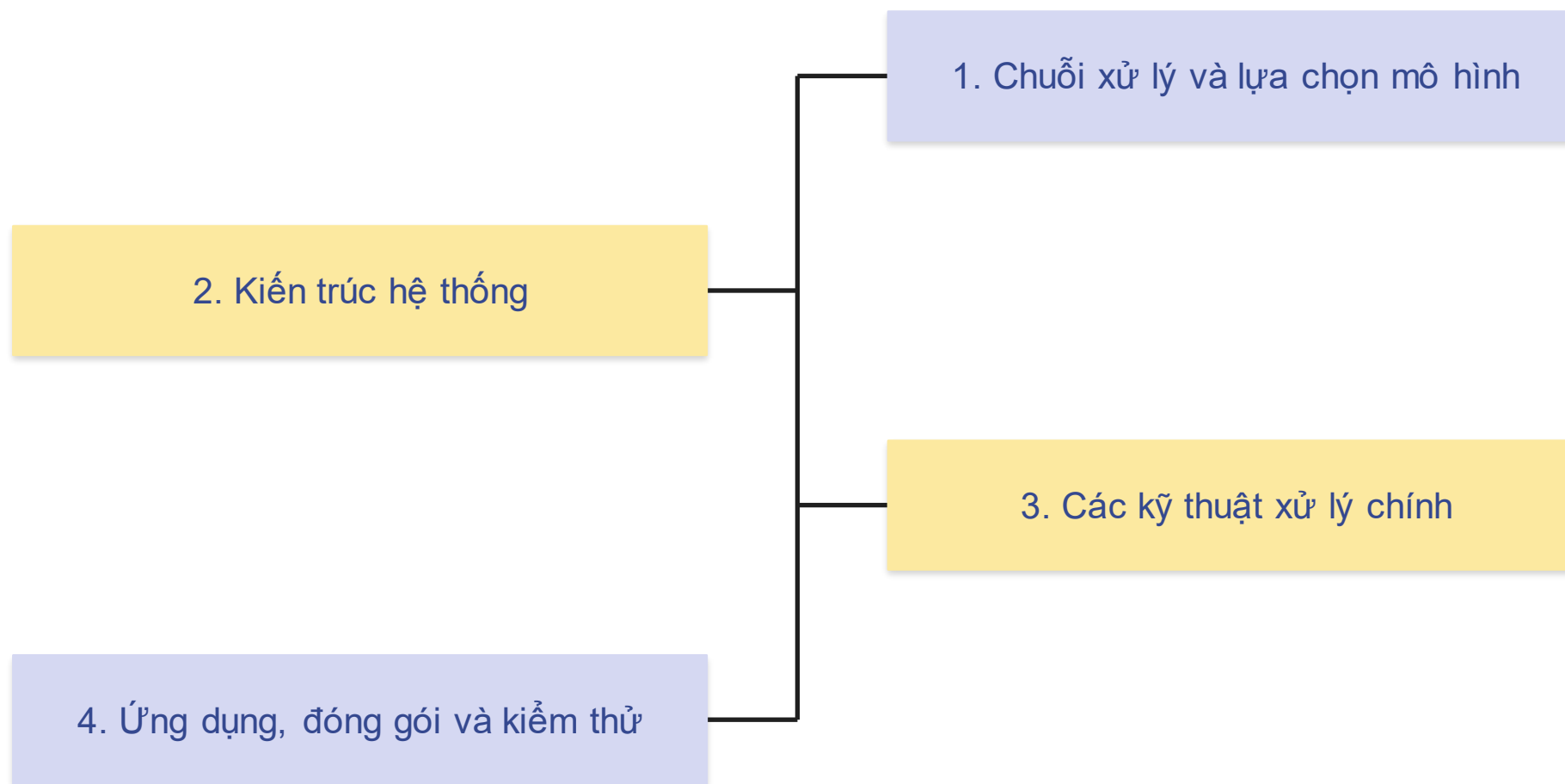
- **Input:** giọng người dùng qua microphone và âm thanh đang phát trên máy (phía bên kia cuộc gọi, video, bài giảng)
- **Output:** phụ đề song ngữ trên màn hình; giọng đọc bản dịch ra tai nghe
- **Ngôn ngữ:** 6 chiều dịch Việt ↔ Anh / Nhật / Trung giản thể
- **Nền tảng:** Windows 11 x64 và macOS 13+ Apple Silicon, đóng gói thành bộ cài
- **Không dùng đám mây** trong lúc dịch; mô hình chỉ tải về một lần

Ngoài phạm vi: không đề xuất mô hình AI mới (đề tài là tích hợp hệ thống); không dịch đồng thời theo từng từ, đơn vị xử lý là một câu sau khi người nói ngắt; không nhân bản giọng; không đưa bản dịch ngược vào phần mềm học.



6 chiều dịch, tiếng Việt là trục

II. Phương pháp thực hiện



II. Phương pháp thực hiện

1. Chuỗi xử lý dịch giọng nói



Vì sao chọn cách chuỗi (thay vì đầu-cuối)

- Đo được lỗi từng khâu: biết bản dịch sai do nghe nhầm hay dịch nhầm
- Thay được từng khâu theo phần cứng: đổi runtime ASR không đọng khâu dịch
- Mỗi khâu đã có mô hình mã nguồn mở đủ tốt, phủ đủ 4 ngôn ngữ

Cái giá phải trả

- Lỗi cộng dồn: từ nghe nhầm được dịch luôn cái sai
- Độ trễ cộng dồn: bốn khâu chạy nối tiếp
- Cả hai được đo cụ thể ở phần III

II. Phương pháp thực hiện

1. Lựa chọn mô hình

Khâu	Mô hình / runtime	Lý do chọn
VAD	Silero VAD [24]	~1 MB, chạy CPU, xử lý cửa sổ 32 ms trong vài chục ms
ASR	Whisper large-v3 / large-v3-turbo [19] trên 3 runtime: whisper.cpp (Vulkan, Metal), faster-whisper (CUDA), MLX (Apple Silicon)	Đa ngôn ngữ sẵn; có bản lượng tử hóa chạy được trên máy cá nhân; mỗi nền tảng một runtime nhanh nhất
MT	NLLB-200 distilled 600M [4]	Một mô hình phủ đủ 6 chiều, dịch trực tiếp Việt ↔ Nhật / Trung không qua tiếng Anh
TTS	sherpa-onnx [12] (vi, en, zh) + Kokoro [9] với OpenJTalk (ja)	Chạy ONNX offline; sherpa-onnx không có front-end tiếng Nhật dùng được nên ghép thêm Kokoro

Ba mức cấu hình sẵn, đổi được ngay trong ứng dụng:

Nhanh

whisper small · trần câu 4,5 s

Cân bằng

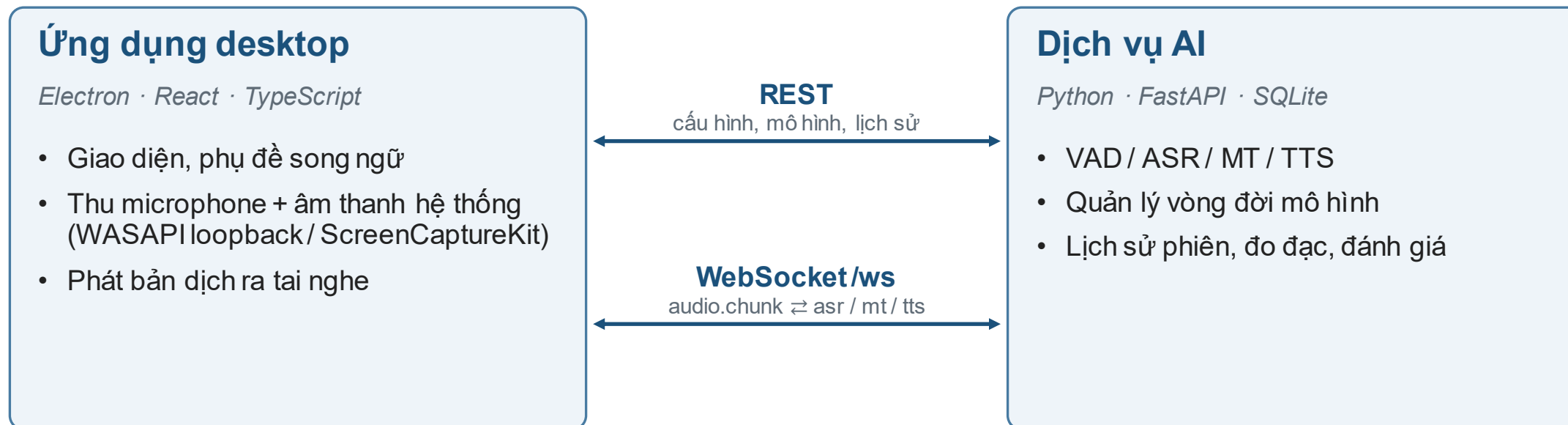
large-v3-turbo q5 · trần câu 6 s

Chất lượng

large-v3 · trần câu 8 s

II. Phương pháp thực hiện

2. Kiến trúc: hai tiến trình trên một máy



Chỉ lắng nghe trên 127.0.0.1: không mở ra mạng LAN, không gọi dịch vụ đám mây trong lúc dịch

Vì sao tách hai tiến trình:

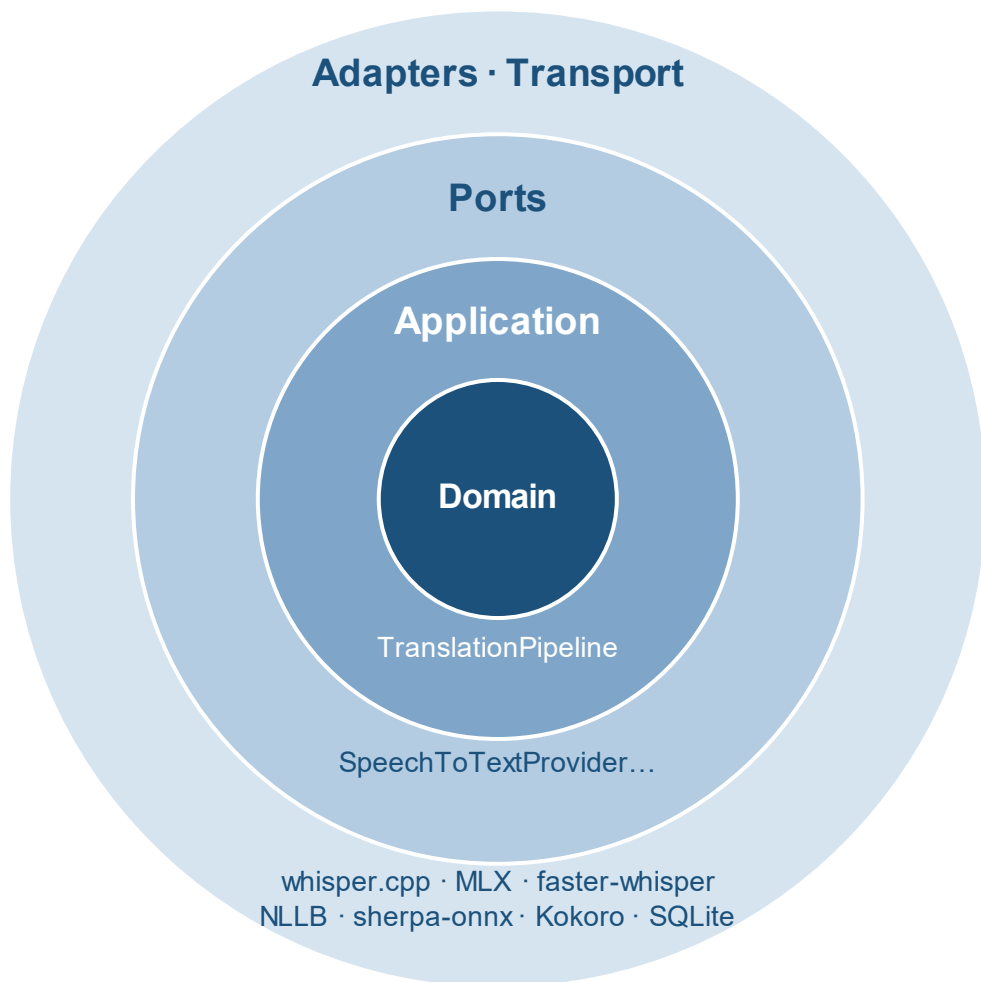
Hệ sinh thái mô hình (whisper.cpp, transformers, sherpa-onnx) nằm ở Python; thu âm thanh hệ thống là thứ Chromium làm sẵn

Mô hình chiếm ~2 GB RAM và chặn CPU hàng giây: chung tiến trình thì giao diện đơ mỗi câu

Dịch vụ lỗi thì cửa sổ vẫn sống để báo lỗi và khởi động lại

II. Phương pháp thực hiện

2. Kiến trúc lục giác (Ports & Adapters) [2]



Một quy tắc: phụ thuộc luôn hướng vào trong

Chuỗi xử lý chỉ biết các port (giao diện trừu tượng), không biết mô hình nào đang chạy phía sau.



Thêm 2 runtime ASR (MLX, faster-whisper)

Mỗi runtime: 1 adapter + 1 dòng đăng ký + 1 dòng preset. Không sửa dòng nào của chuỗi xử lý.



TTS tiếng Nhật hồng giữa chừng

Thêm adapter Kokoro và LanguageRoutedTts chọn engine theo ngôn ngữ đích; chuỗi xử lý vẫn chỉ thấy một TTS.



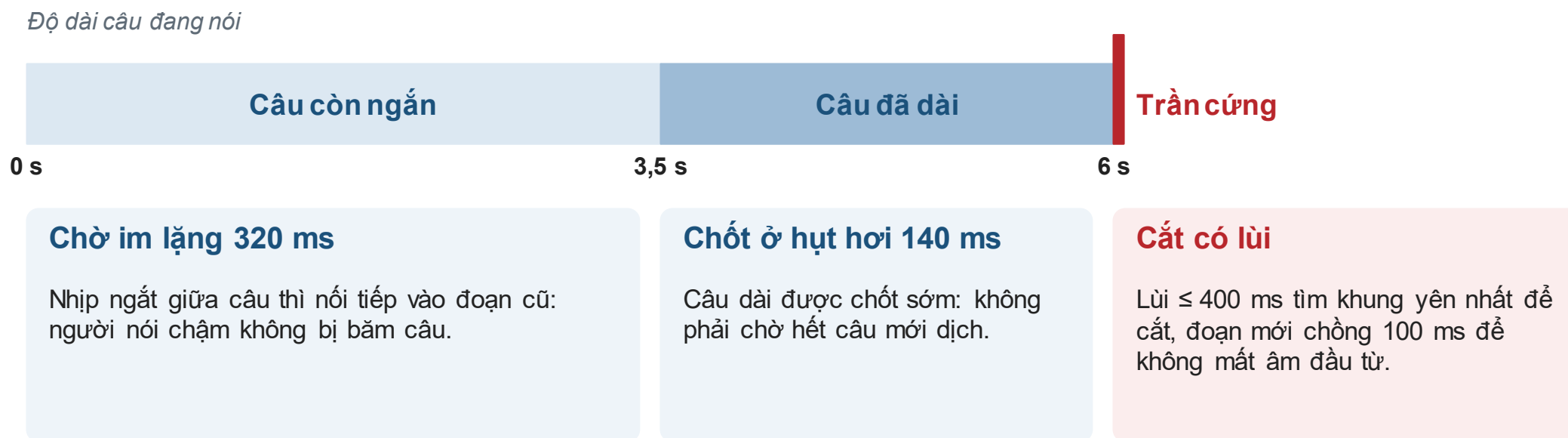
Kiểm thử không cần mô hình thật

Thay adapter bằng mô hình giả: 327 bài kiểm thử dịch vụ chạy trong vài giây, tắt định.

II. Phương pháp thực hiện

3. Bộ tách câu: khi nào một câu kết thúc?

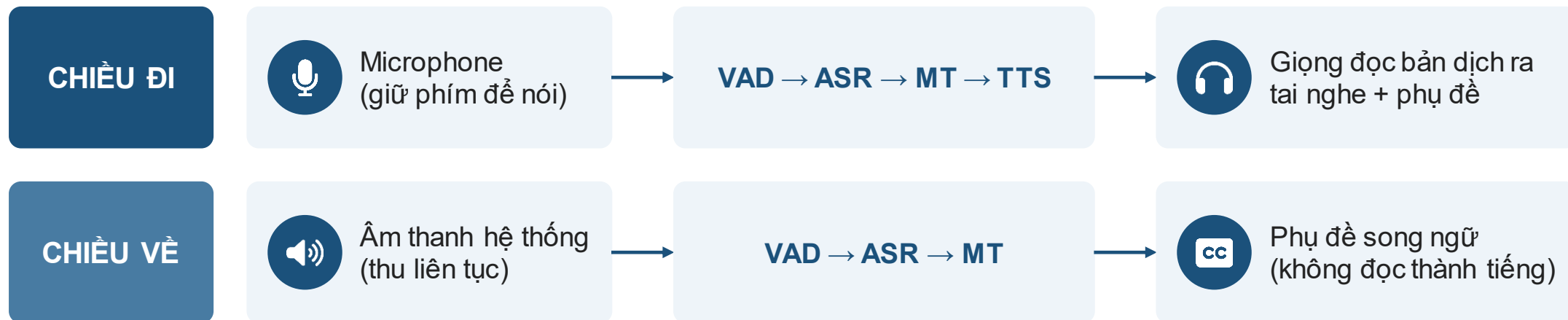
Vấn đề: lời nói không có dấu chấm câu. Phiên bản đầu dùng một ngưỡng im lặng 300 ms và trần 20 s: người nói liền mạch phải chờ tới 20 s mới có bản dịch (GVHD góp ý, 19/08/2026).



Kết quả trên giọng thật (III.4): độ dài câu p90 của bản tin đọc liên tục **12,5 s** → **5,9 s**; hội thoại tự nhiên gần như không bị cắt giữa câu. Tham số của preset Cân bằng; mỗi preset có bộ tham số riêng.

II. Phương pháp thực hiện

3. Hai chiều dịch và chống vòng lặp âm thanh



Chống vòng lặp

Phát bản dịch ra loa thì chính nó bị thu lại và dịch tiếp. Ứng dụng bỏ mọi khung âm thanh hệ thống thu trong lúc đang phát bản dịch, giao diện hiện "Tạm ngưng thu".



Nhả phím là chốt câu

Ngừng gửi âm thanh thì VAD không bao giờ thấy khoảng lặng. Thêm `VadStream.flush()` vào port VAD để chốt câu ngay khi nhả phím.



Chặn hai lần

Ngoài lúc giữ phím, ứng dụng không gửi và dịch vụ cũng bỏ khung đến, không tin phía client.

II. Phương pháp thực hiện

3. Nạp mô hình theo yêu cầu và GPU trên Windows

45 s $\rightarrow$ 0,4 s

Thời gian mở dịch vụ

Không nạp gì lúc khởi động; mọi đường cần mô hình đi qua một cửa `ModelManager.ensure_loaded()`. Chọn preset không tải gì.

17,6 s $\rightarrow$ 0,44 s

ASR cho 3 s âm thanh trên bản cài Windows

`whisper.cpp` dựng với Vulkan thay vì chạy CPU (17.573 ms $\rightarrow$ 437 ms).

+18 MB

thay vì ~1 GB nếu đóng gói CUDA

Vulkan có sẵn trong driver NVIDIA, AMD, Intel: một bộ cài chạy GPU trên mọi hãng. Ý tưởng từ TranscriptionSuite [10].

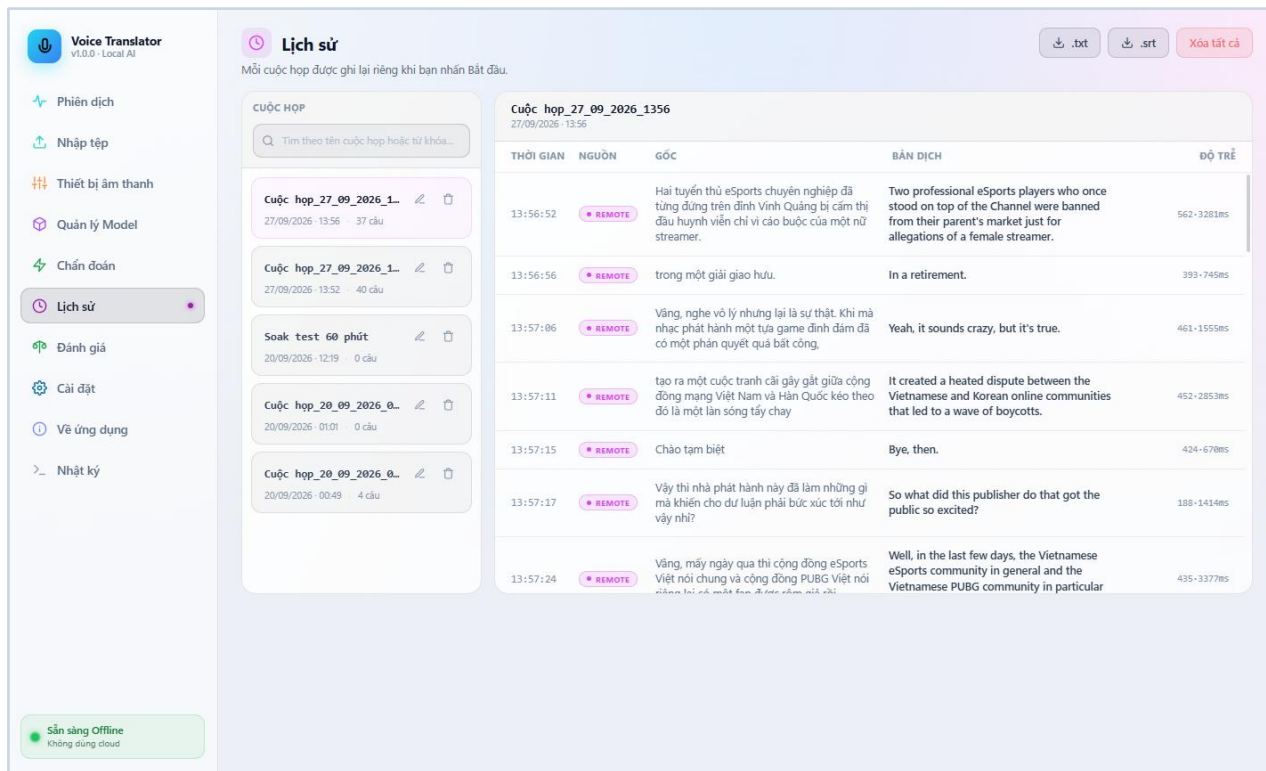
9,9 s $\leftrightarrow$ 0,17 s

GPU tích hợp so với GPU rời

`whisper.cpp` lấy GPU đầu tiên, trên laptop hai card là Iris Xe. Adapter tự liệt kê thiết bị và chọn card rời.

II. Phương pháp thực hiện

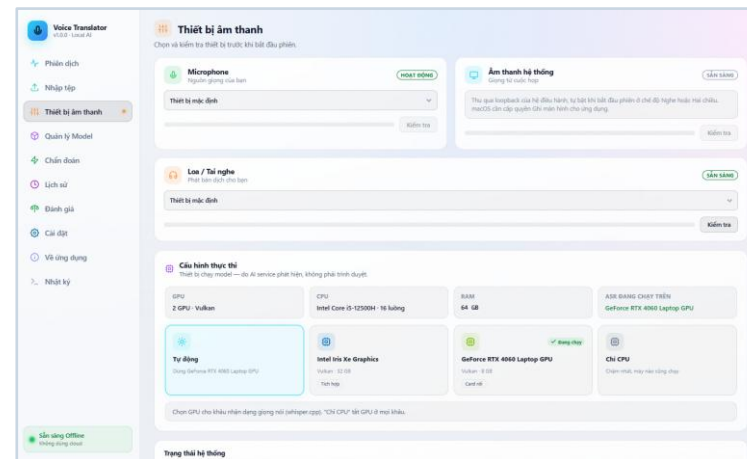
4. Ứng dụng desktop



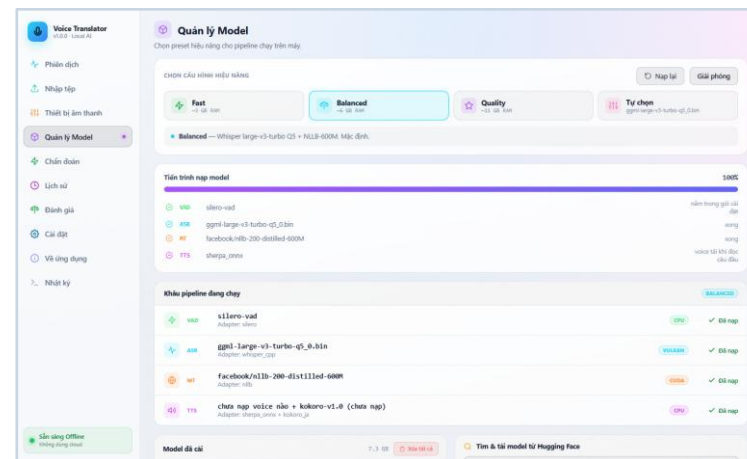
Lịch sử: bản gốc và bản dịch của cả hai chiều

8 màn hình: Phiên dịch · Nhập tệp · Thiết bị âm thanh · Quản lý Model · Chẩn đoán · Lịch sử · Đánh giá · Cài đặt

Ngoài đề cương: lịch sử có tìm kiếm và tắt lưu được, nhập tệp âm thanh/video, duyệt bản dịch trước khi đọc, chấm điểm trong ứng dụng, tách người nói cho tệp nhập



Thiết bị âm thanh và GPU dịch vụ nhìn thấy



Quản lý mô hình, preset và tiến độ nạp

II. Phương pháp thực hiện

4. Đóng gói và kiểm thử

Bộ cài tự chứa: máy người dùng không cần Python, Node hay trình quản lý gói

.exe 371 MB

Windows 11 x64

.dmg 592 MB

macOS Apple Silicon

Kiểm tra bộ cài Windows theo bốn tầng, từ trong ra ngoài:

Tầng	Kiểm gì	Kết quả
1	Bộ Python đóng gói	Nạp PyTorch, sherpa-onnx, pywhispercpp
2	Dịch vụ chạy từ bộ đó	Nạp 4 khâu trong 34 s, ASR báo Vulkan
3	Bộ cài .exe thật	Dịch vụ tự lên sau 12 s, tắt thì nhả cổng
4	Tốc độ trên bản cài	ASR 3 s âm thanh: 17.573 → 437 ms

Ba lớp kiểm thử tự động

327

pytest

Dịch vụ: pipeline, tách câu, lọc câu bịa, API, lịch sử

69

vitest + jsdom

Giao diện trên dịch vụ giả

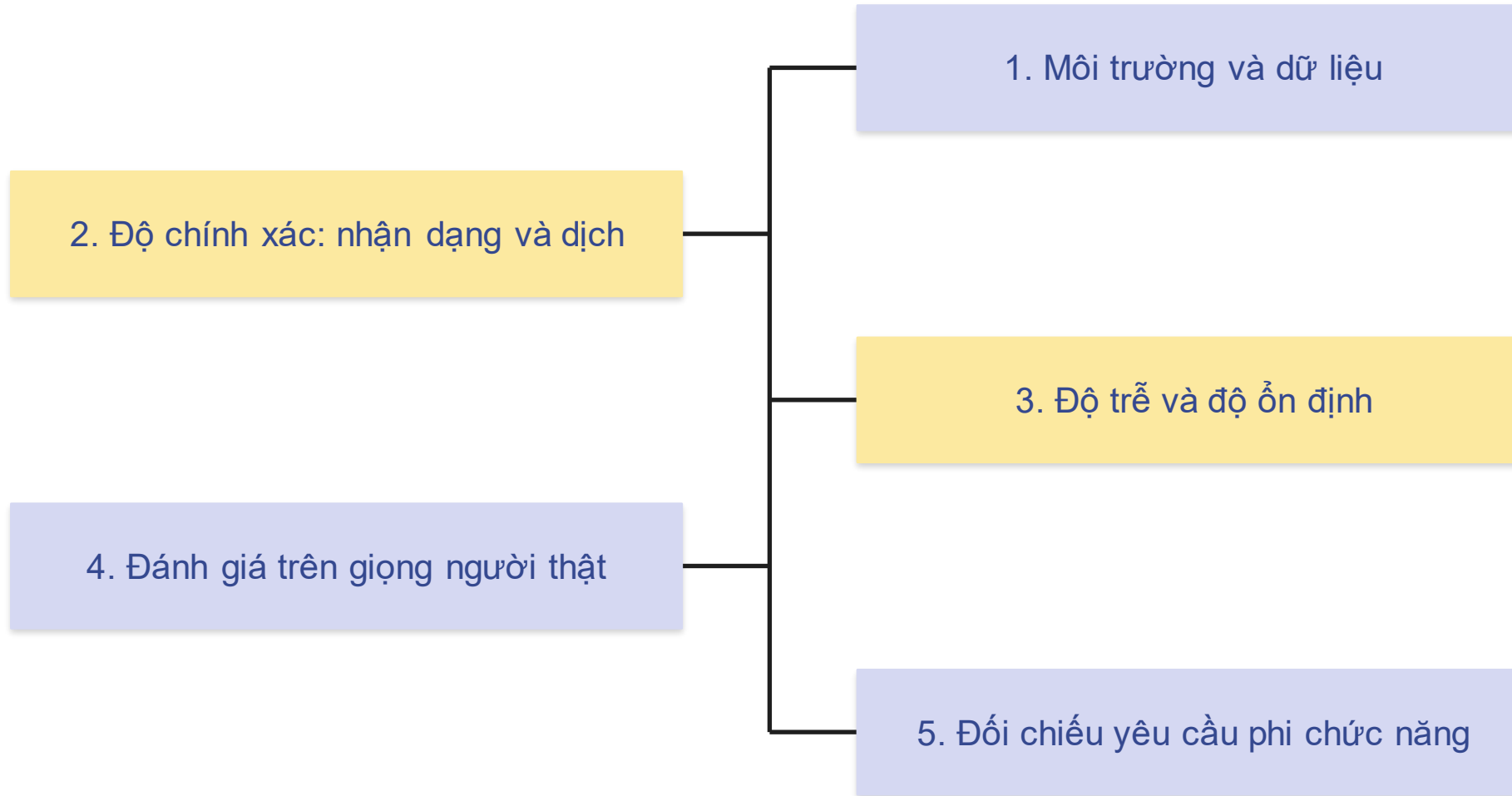
11

Playwright

App Electron đã dựng + dịch vụ thật + tải mô hình thật

Hai lớp đầu dùng mô hình giả: chạy vài giây, tắt định, không cần mạng.

III. Kết quả thực nghiệm



III. Kết quả thực nghiệm

1. Môi trường và dữ liệu thử nghiệm



macOS

Apple M4 · macOS 26.6
ASR: MLX/Metal · MT: PyTorch/MPS · TTS: CPU



Windows

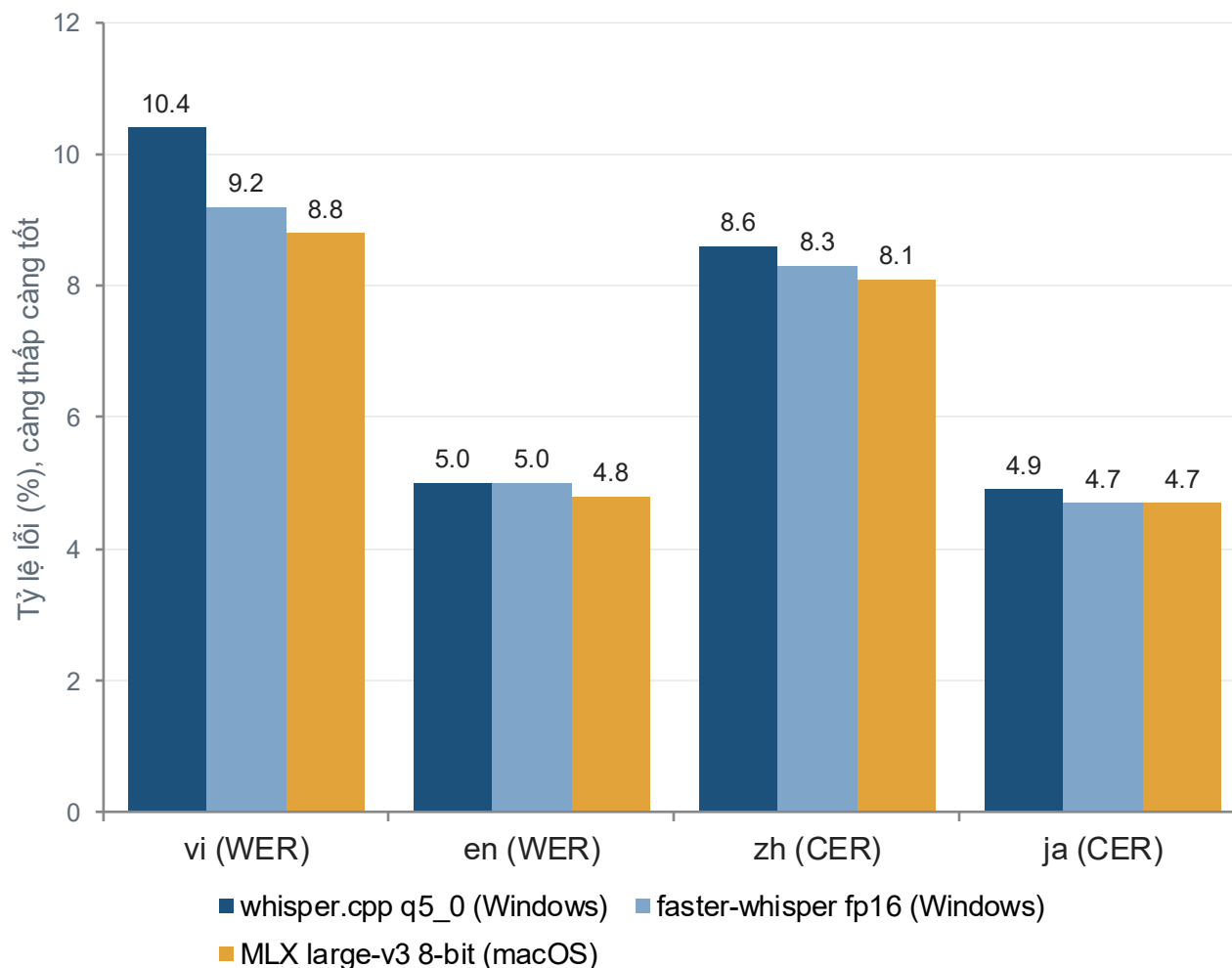
i5-12500H · RTX 4060 Laptop 8 GB + Iris Xe · 64 GB RAM
ASR: whisper.cpp/Vulkan · MT: PyTorch/CUDA · TTS: CPU

Dữ liệu	Dùng cho	Quy mô
FLEURS test [3], âm thanh	Nhận dạng (ASR)	3.099 bản thu, 10,22 giờ: vi 857 · en 647 · zh 945 · ja 650
FLEURS test, văn bản	Dịch máy (MT)	2.022 cặp câu, sáu chiều
FLEURS test, âm thanh	Độ trễ cả chuỗi	50 mẫu × 6 chiều
Âm thanh theo nhịp thật	Chạy liên tục	60 phút, 877 câu
3 bản ghi YouTube	Tách câu	Bản tin đọc · phỏng vấn tự phát · TEDx
25 đoạn phỏng vấn	ASR + MT giọng thật	149 s tiếng nói, câu tham chiếu nghe và sửa tay

Nguyên tắc đo: đo nóng (sau khi nạp mô hình); khâu dịch đo trên văn bản tham chiếu để lỗi ASR không cộng vào điểm MT; giải mã tắt định nên chạy lại ra đúng WER cũ. Mọi kết quả gốc lưu JSON cùng mã nguồn.

III. Kết quả thực nghiệm

2. Nhận dạng tiếng nói trên FLEURS (3.099 bản thu)



- Tiếng Việt khó gần gấp đôi tiếng Anh: WER **8,8–10,4%** so với 4,8–5,0%. FLEURS là giọng đọc nên đây là mức sàn.
- Cùng large-v3-turbo, bản q5_0 kém bản fp16 1,2 điểm WER tiếng Việt, tiếng Anh không đổi, nhưng nhận dạng nhanh gấp đôi (RTF 0,018 so với 0,034).
- Tiếng Trung và tiếng Nhật dùng CER vì không tách từ bằng khoảng trắng, như bài báo FLEURS.
- Ba runtime không dùng chung tệp mô hình, nên ba cột là ba mô hình khác nhau và bảng này so ba cấu hình.

III. Kết quả thực nghiệm

2. Dịch máy trên FLEURS (NLLB-200 distilled 600M)

Chiều	Số câu	spBLEU	chrF++	COMET
vi→en	347	35,79	57,10	0,853
en→vi	347	37,13	54,69	0,851
vi→zh	346	17,15	16,14*	0,773
zh→vi	346	22,33	42,77	0,821
vi→ja	318	10,96	19,88*	0,824
ja→vi	318	19,91	40,19	0,819

* chrF++ của đích zh/ja không so ngang được (không có khoảng trắng); COMET dùng Unbabel/wmt22-comet-da [21]

spBLEU và COMET xếp hạng khác nhau

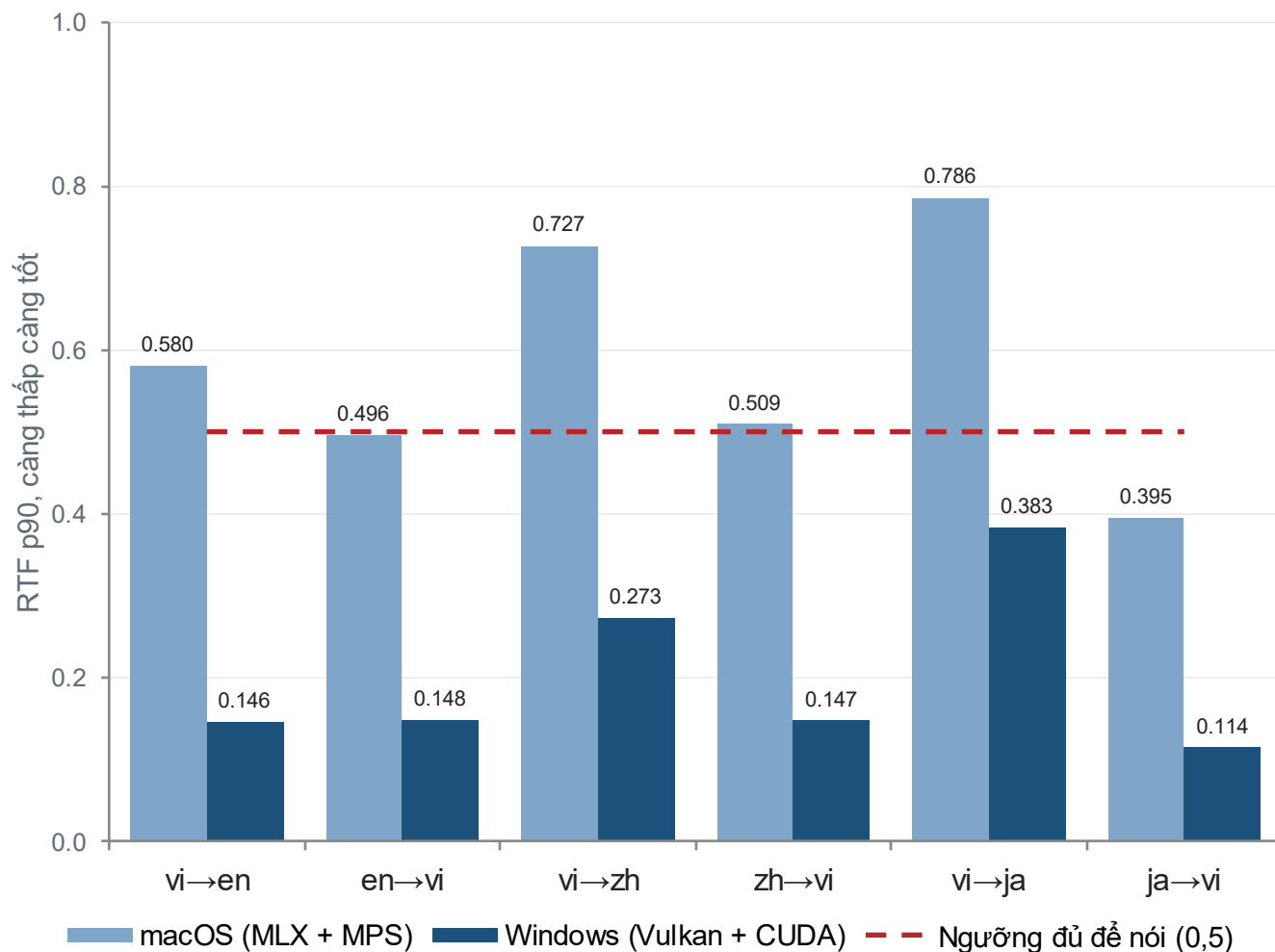
Theo spBLEU, vi→ja tệ nhất (10,96). Theo COMET, vi→ja (0,824) lại cao hơn vi→zh (0,773). spBLEU khớp chuỗi bề mặt nên phạt nặng ngôn ngữ khác hệ chữ; COMET chấm theo nghĩa.

Chiều yếu nhất là vi→zh, nơi cả hai độ đo đồng ý. Vì hai độ đo có thể xếp hạng ngược nhau, đồ án báo cáo cả spBLEU lẫn COMET.

COMET 0,77–0,85 trên cả sáu chiều. Không lấy trung bình sáu chiều vì độ khó rất khác nhau.

III. Kết quả thực nghiệm

3. Độ trễ: hệ số thời gian thực (RTF p90)

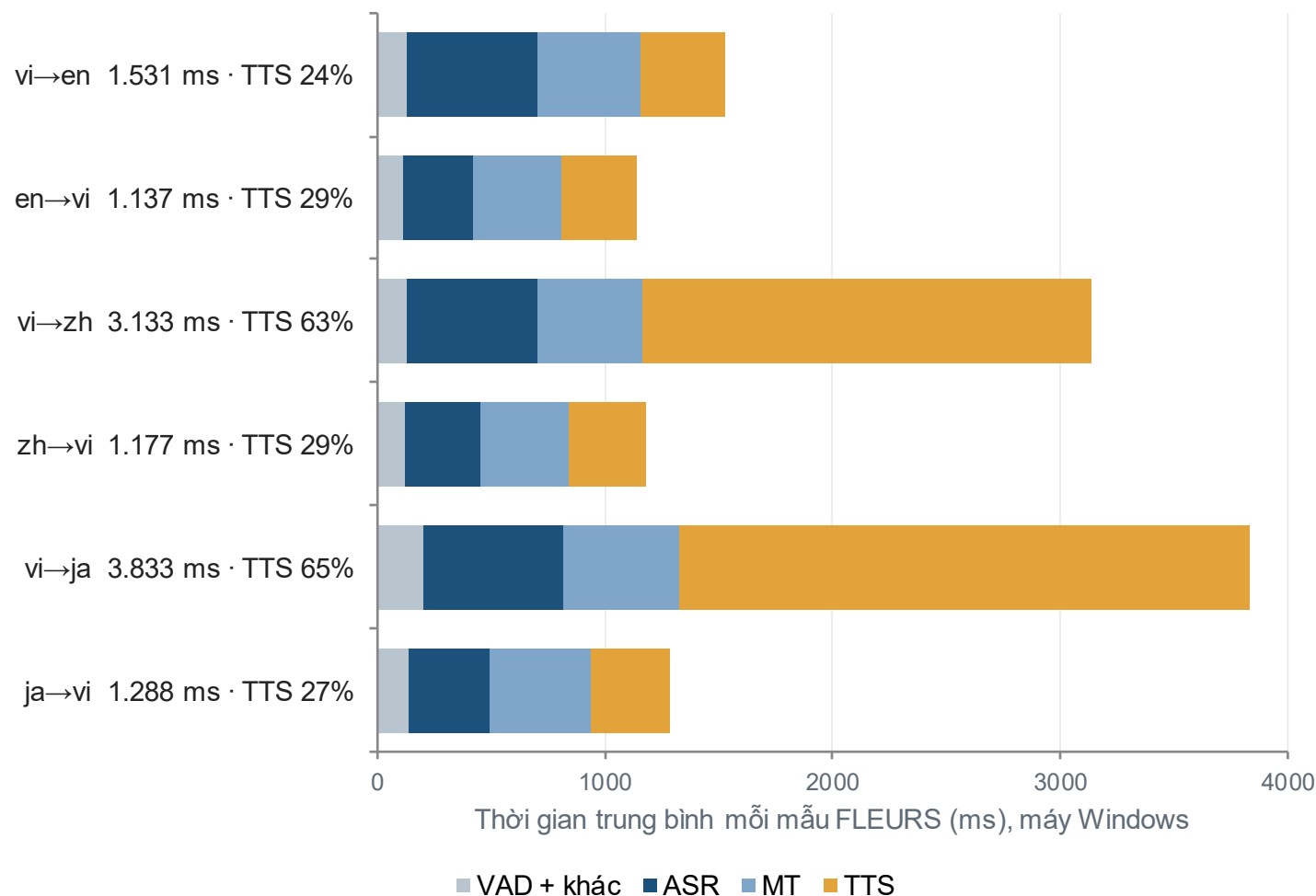


RTF = thời gian xử lý / thời lượng âm thanh
(50 mẫu mỗi chiều, đủ chuỗi VAD → ASR → MT → TTS)

- RTF p90 dưới 1 ở cả sáu chiều trên cả hai máy, tức hệ thống theo kịp luồng nói liên tục.
- Với ngưỡng 0,5, máy Windows đạt cả sáu chiều (xấu nhất vi→ja 0,383). Máy macOS đạt hai chiều en→vi và ja→vi; zh→vi trượt sát mép ở 0,509.
- Thời gian chờ chốt câu là 200–256 ms, chỉ phụ thuộc ngôn ngữ nguồn và giống nhau trên hai máy.

III. Kết quả thực nghiệm

3. Điểm nghẽn chuyển sang TTS khi có GPU



- So với máy macOS, GPU làm ASR nhanh khoảng 8 lần (whisper.cpp/Vulkan) và MT nhanh 2–2,5 lần (NLLB/CUDA).
- TTS vẫn chạy CPU nên gần như giữ nguyên, và thành khâu chậm nhất ở hai đích zh/ja: **63% và 65%** toàn chuỗi.
- **Chỗ cần tối ưu tiếp theo là TTS cho đích tiếng Nhật và tiếng Trung.**

III. Kết quả thực nghiệm

3. Câu ngắn so với mục tiêu và chạy liên tục 60 phút

Một câu 3 giây, máy macOS, preset Cân bằng

Chiều	VAD	ASR	MT	TTS	Tổng
vi→en	46	1.060	569	141	1.819
en→vi	46	991	579	148	1.766
vi→ja	48	1.056	460	772	2.338
vi→zh	46	1.038	558	969	2.613
Mục tiêu		< 1.500	< 1.000	< 1.500	< 4.000

Đơn vị: ms. Cả bốn mốc đều đạt ở cả bốn chiều. Trên bản cài Windows, cùng phép đo cho ASR 362–480 ms.

Sau một giờ, bộ nhớ và độ trễ gần như đứng yên: không có dấu hiệu rò rỉ đáng kể.

Chạy liên tục 60 phút (Windows)

877 / 0

câu thành công / lỗi

+12 MB

bộ nhớ RSS (1.110 → 1.122 MB)

+1,8 ms

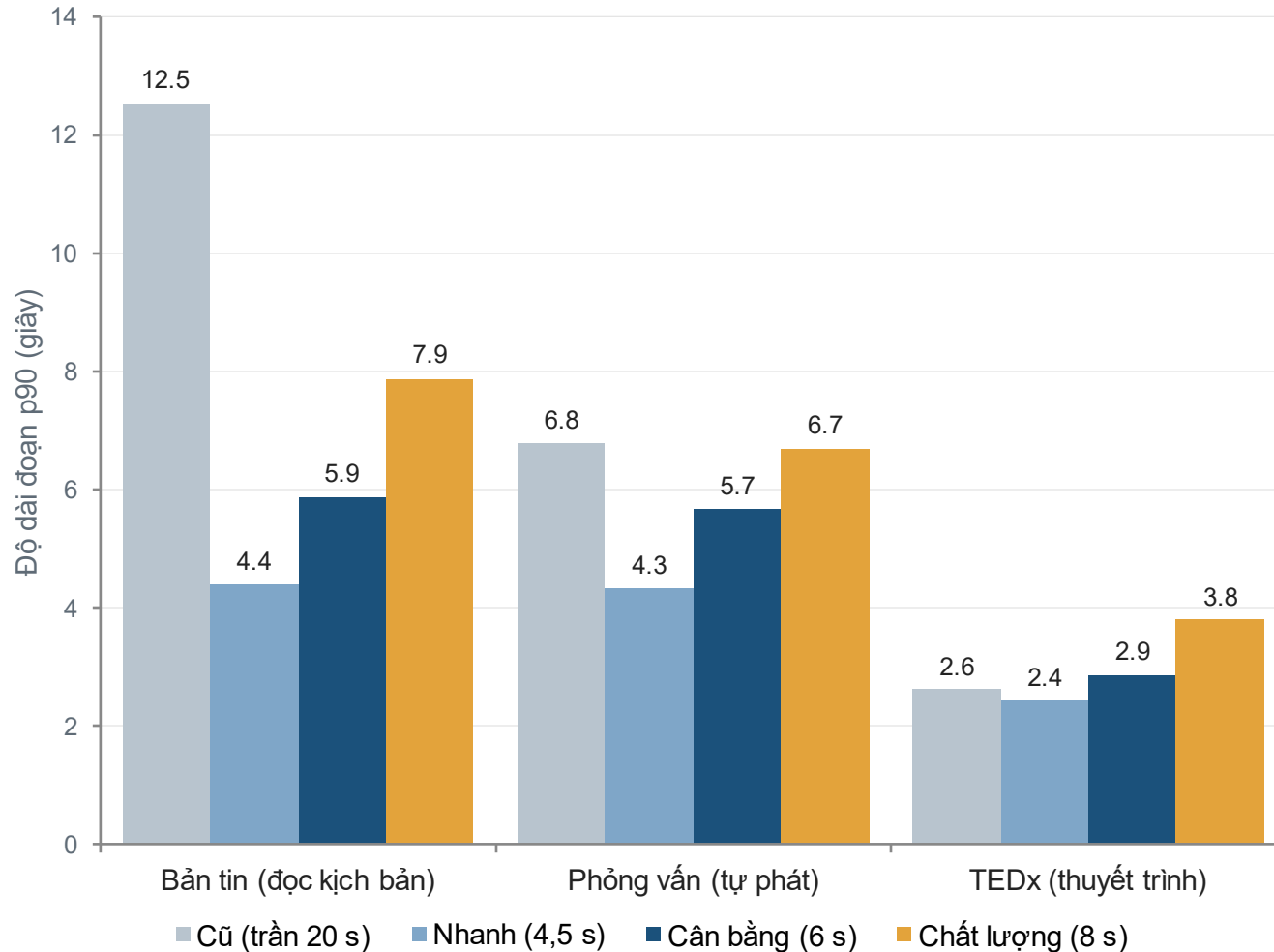
độ trễ 10% câu cuối so với 10% câu đầu

443 ms

độ trễ p50 (p95 541 ms), không mất kết nối

III. Kết quả thực nghiệm

4. Tách câu trên ba loại giọng người thật



- Với tham số cũ, 10% số câu của bản tin dài hơn 12,5 s, đúng vấn đề GVHD chỉ ra. Preset Cân bằng kéo xuống 5,9 s.
- Thời gian chờ chốt gần như không đổi (0,20–0,36 s), nên phần cải thiện đến từ trần độ dài.
- Cái giá là cắt giữa câu: preset Cân bằng cắt cứng 30% số đoạn bản tin, 10% ở phỏng vấn và 0% ở TEDx, nên hội thoại tự nhiên gần như không bị ảnh hưởng.

III. Kết quả thực nghiệm

4. Nhận dạng và dịch trên hội thoại tự phát

WER tiếng Việt, cùng mô hình, cùng máy

10,4%

FLEURS, 857 câu



24,6%

phỏng vấn thật, 25 đoạn

Giọng nói tự phát cho WER gấp 2,4 lần giọng đọc

Hai lỗi của khâu dịch mà giọng đọc không bắt được

Vòng lặp mất kiểm soát

2 / 75 lượt, đều sang tiếng Nhật. Câu mở đầu bằng từ đệm ("dạ đúng ạ...") thành "そうだ。" lặp hơn 50 lần. NLLB chỉ có trần 256 token, và TTS sẽ đọc to toàn bộ.

Lỗi chủ yếu: thuật ngữ tiếng Anh đọc theo giọng Việt

- "string" xuất hiện 18 lần, Whisper nghe đúng **1 lần**; nghe thành "stream" 6 lần, "trên" 5, "chuyên" 3.
- Phụ đề tự sinh của YouTube sai đúng kiểu đó: giới hạn chung của nhận dạng đa ngôn ngữ khi chen từ tiếng Anh vào câu tiếng Việt.

Dịch nghĩa đen thuật ngữ

"spring" thành 春天 (mùa xuân) ở đích tiếng Trung, "春のバツファー" ở đích tiếng Nhật. NLLB không có khái niệm thuật ngữ cần giữ nguyên.

Quy mô nhỏ (1 bản ghi, 25 đoạn), nên chỉ cho biết độ lớn của khoảng cách; số chính vẫn là các bảng FLEURS.

III. Kết quả thực nghiệm

5. Đối chiếu với yêu cầu phi chức năng

Yêu cầu	Kết quả	Đánh giá
Phát hiện kết thúc câu < 700 ms	Chờ chốt 200–256 ms (FLEURS), 0,20–0,36 s (giọng thật)	Đạt
ASR một câu ngắn < 1.500 ms	991–1.060 ms (macOS), 362–480 ms (Windows), câu 3 s	Đạt
Dịch < 1.000 ms; TTS < 1.500 ms	MT 460–579 ms; TTS 141–969 ms	Đạt
Tổng chiều đi < 4 s; phụ đề chiều về < 3 s	1,8–2,6 s; ASR + MT < 1,2 s trên Windows	Đạt
Chạy 60 phút, bộ nhớ không tăng đều	877 câu, 0 lỗi, +12 MB	Đạt
Một mô hình lỗi không làm mất cả phiên	Câu lỗi chuyển Error, phiên chạy tiếp (kiểm thử đơn vị)	Đạt
Offline sau khi cài mô hình	Không có lời gọi mạng trên đường dịch; tài liệu API đóng gói sẵn	Đạt
Không gửi, không lưu âm thanh; tắt được lịch sử	Âm thanh chỉ qua 127.0.0.1; lịch sử chỉ lưu văn bản	Đạt
Windows 11 và macOS Apple Silicon	Có bộ cài và số đo trên cả hai	Đạt
Đổi runtime không sửa chuỗi xử lý	Thêm MLX, faster-whisper không sửa dòng nào	Đạt

Chưa đo: "người dùng mới hoàn thành cấu hình trong 5 phút" (cần nhóm người dùng thử). Offline: đạt theo thiết kế, chưa có bài thử ngắt mạng tự động.

IV. Kết luận

1. Kết quả đạt được



Chuỗi xử lý bốn khâu, thay được từng khâu

6 chiều dịch Việt ↔ Anh / Nhật / Trung; 3 runtime ASR thay thế nhau, thêm vào không sửa chuỗi xử lý.



Hai nguồn âm thanh, không vòng lặp

Micro và âm thanh hệ thống thu đồng thời, tách bạch; bản dịch đang phát không bị thu ngược.



Chạy trên hai nền tảng

Bộ cài tự chứa cho Windows 11 và macOS Apple Silicon; whisper.cpp chạy GPU qua Vulkan với +18 MB.



Đánh giá bằng độ đo chuẩn trên FLEURS

WER vi 8,8–10,4%, en 4,8–5,0%; CER zh/ja < 9%; COMET 0,77–0,85; Windows: cả 6 chiều RTF $p_{90} \leq 0,5$; 60 phút không lỗi.



Biết con số lạc quan tới đâu

Giọng thật: WER vi 24,6% so với 10,4%; bộ tách câu kéo câu p90 từ 12,5 s xuống 5,9 s.

Đóng góp: kiến trúc thay và đo được từng khâu; bộ đánh giá tái lập được, mọi số liệu có tệp kết quả gốc; các phát hiện chỉ có khi chạy máy thật.

IV. Kết luận

2. Hạn chế và hướng phát triển

Hạn chế

Thuật ngữ tiếng Anh trong câu tiếng Việt: "string" đúng 1/18 lần



NLLB rơi vào vòng lặp (2/75 lượt), dịch nghĩa đen thuật ngữ



TTS đích ja/zh chậm, chạy CPU, ~2/3 thời gian chuỗi



Đo giọng thật quy mô nhỏ; chưa đo trên máy chỉ có CPU



NLLB-200 giấy phép phi thương mại (CC-BY-NC-4.0)



Hướng phát triển

Danh sách thuật ngữ: gợi ý cho Whisper (initial_prompt) và giữ nguyên khi dịch

Chặn lặp bằng tham số sinh hoặc bộ lọc sau dịch, rồi chạy lại bộ đánh giá MT

Đưa TTS lên GPU cho hai đích tiếng Nhật và tiếng Trung

Nhiều bản ghi, nhiều chủ đề, hai người nghe độc lập; thêm cấu hình máy yếu

Thử mô hình dịch khác trên cùng bộ đánh giá, như đã làm với ASR

- [2] A. Cockburn, "**Hexagonal architecture**," 2005. <https://alistair.cockburn.us/hexagonal-architecture/>
- [3] A. Conneau et al., "**FLEURS: Few-shot learning evaluation of universal representations of speech**," in Proc. IEEE SLT, 2022.
- [4] M. R. Costa-jussà et al. (NLLB Team), "**No language left behind: Scaling human-centered machine translation**," arXiv:2207.04672, 2022.
- [7] Google, "**Speech translation in Google Meet**," Google I/O, May 2025.
- [9] hexgrad, "**Kokoro-82M**," Hugging Face, 2025.
- [10] homelab-00, "**TranscriptionSuite: A fully local and private speech-to-text application**," GitHub, 2026.
- [12] k2-fsa, "**sherpa-onnx: Speech-to-text, text-to-speech and audio processing with ONNX Runtime**," 2026.
- [19] A. Radford et al., "**Robust speech recognition via large-scale weak supervision**," in Proc. ICML, 2023, pp. 28492–28518.
- [21] R. Rei et al., "**COMET-22: Unbabel-IST 2022 submission for the metrics shared task**," in Proc. WMT, 2022, pp. 578–585.
- [22] royshil, "**LocalVocal: Local live captions & translation on-the-go (OBS plugin)**," GitHub.
- [23] Seamless Communication et al., "**SeamlessM4T: Massively multilingual & multimodal machine translation**," arXiv:2308.11596, 2023.
- [24] Silero Team, "**Silero VAD: Pre-trained enterprise-grade voice activity detector**," GitHub, 2026.

DEMO

Phiên dịch hai chiều Anh ↔ Việt · chống vòng lặp · duyệt trước
khi đọc · lịch sử · chạy không cần mạng

The background features a central white area with soft, wavy borders in shades of light blue and dark blue. Scattered throughout are small, dried, golden-brown leaves and petals, some of which are clustered into two larger, more defined branches on the left and right sides.

**Cảm ơn quý Thầy Cô
đã lắng nghe**

Q&A